

LINQ Where Method in C# with Examples

In this article, I will discuss **LINQ Where Method in C#** with Examples. Please read our previous article discussing the [LINQ SelectMany Projection Method in C#](#) with Examples. The standard query operator Where comes under the Filtering Operators category in LINQ. At the end of this article, you will understand the following concepts.

1. [What is Filtering?](#)
2. [What are the Filtering Methods available in LINQ?](#)
3. [The “Where” Method and its Examples using both Method and Query Syntax](#)
4. [What is a Predicate?](#)
5. [When to Use Where Method in LINQ?](#)

What is Filtering?

Filtering is nothing but the process of getting only those elements from a data source that satisfies the given condition. It is also possible to fetch the data from a data source with more than one condition as per our business requirements. For example:

1. Employees having a salary greater than 50000.
2. Students Having Marks greater than 80% from a particular batch.
3. Employees having experience of more than 6 Years and the department is IT, etc.

What are the Filtering Methods Available in LINQ?

Two methods provided by LINQ in C# are used for filtering. They are as follows:

1. **Where Method**
2. **OfType Method**

LINQ Where Filtering Operator or Method:

The LINQ Where Method is used for filtering collections based on a predicate that takes each element in the collection and returns a boolean value. If the function returns true for an element, it is included in the result; otherwise, it's excluded.

That means the Where method filters a sequence (such as an array or a list) based on a predicate. A predicate is a function that takes an element of the sequence as input and returns a boolean value (true or false). The method returns all elements for which the predicate returns true.

The “Where” Method always expects at least one condition, and we can specify the condition(s) using predicates. The conditions can be written using the **=, >=, <=, &&, ||, >, <, etc.** symbols. Two overloaded versions of the “Where” Method are available in LINQ. They are as follows:

```

public static IEnumerable<TSource> Where<TSource>(
    this IEnumerable<TSource> source,
    Func<TSource, bool> predicate);

public static IEnumerable<TSource> Where<TSource>(
    this IEnumerable<TSource> source,
    Func<TSource, int, bool> predicate);

```

As you can see in the above signatures, the methods are implemented as extension methods on the `IEnumerable<T>` interface. The methods accept a predicate as a parameter. So, let us first understand what a predicate is.

What is a Predicate in C#?

A Predicate is nothing but a function, or technically, you can say a delegate used to test every element for a given condition. Let us understand this with an example. In the below example, the Lambda expression (`num => num > 5`) runs for each and every element present in the “`intList`” collection. Then, it will check whether the number is greater than 5. If the number value is greater than 5, then a boolean value `true` is returned; otherwise, it is `false`. In the below example, I am showing both Method and Query Syntax.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<int> intList = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 }
            //Method Syntax
            IEnumerable<int> filteredData = intList.Where(num => num > 5);
            //Query Syntax
            IEnumerable<int> filteredResult = from num in intList
                                             where num > 5
                                             select num;

            foreach (int number in filteredData)
            {
                Console.WriteLine(number);
            }
            Console.ReadKey();
        }
    }
}

```

When we execute the above program, it will give us the following output.

```
6
7
8
9
10
```

Visual Studio Intelligence shows the following when we hover the mouse over the WHERE extension method in the above example.

//Method Syntax

```
IEnumerable<int> filteredData = intList.Where(num => num > 5);
```

(extension) IEnumerable<int> IEnumerable<int>.Where<int>(Func<int,bool> predicate) (+ 1 overload(s))
Filters a sequence of values based on a predicate.

Exceptions:

System.ArgumentNullException

As you can see in the above image, the Predicate (**Func<int, bool>** predicate) expects one input parameter of type integer, and it returns a boolean value. The Func is a generic delegate in C# that takes one or more input parameters and returns one out parameter. The last parameter is considered the return value. The return type is mandatory, but the input parameter is not.

If you are new to Generic Delegate, I strongly recommend reading the following article, which discusses Generic Delegates in C# with examples.

<https://dotnettutorials.net/lesson/generic-delegates-csharp/>

The lambda expression we passed to the Where extension method in the above example operates on integer data type and should return a boolean value; otherwise, we will get a compile-time error. So the following line of code from the above example

```
IEnumerable<int> filteredData = intList.Where(num => num > 5);
```

It can be rewritten as shown below

```
Func<int, bool> predicate = i => i > 5;
```

```
IEnumerable<int> filteredData = intList.Where(predicate);
```

It should give the same output as expected. You can also create a separate function, as shown below, which works as expected.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
```

```

{
    List<int> intList = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 }
    //Method Syntax
    IEnumerable<int> filteredData = intList.Where(num => CheckNumber(r
foreach (int number in filteredData)
{
    Console.WriteLine(number);
}
Console.ReadKey();
}
public static bool CheckNumber(int number)
{
    if (number > 5)
    {
        return true;
    }
    else
    {
        return false;
    }
}
}
}
}

```

Note: It's important to note that the Where method uses deferred execution. This means the filtering is not actually performed when the Where method is called. Instead, the filtering happens when you iterate over the filtered collection, like when using a for each loop or converting it to a list or an array.

Example to Understand the Second Overloaded Version of the Where Method in LINQ:

In the second overloaded version of the “Where” extension method, the generic Func delegate’s second parameter represents the source element’s index position. That means the first parameter is the element (the type is based on the data type of the data source), the second parameter is the index position (the data type is int) of the element in the data source, and the third parameter is the return value (the data type is bool).

```

public static IEnumerable<TSource> Where<TSource>(
    this IEnumerable<TSource> source,
    Func<TSource, int, bool> predicate);

```

Let us see an example to understand this overloaded version of the Where Method. Here, we need to filter only the odd numbers, i.e., those not divisible by 2. Along with the numbers, we also need to fetch the index position of the number. The index is 0 based.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program

```

```

{
    static void Main(string[] args)
    {
        List<int> intList = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 }
        //Method Syntax
        var OddNumbersWithIndexPosition = intList.Select((num, index) => new
        {
            Numbers = num,
            IndexPosition = index
        }).Where(x => x.Numbers % 2 != 0)
        .Select(data => new
        {
            Number = data.Numbers,
            IndexPosition = data.IndexPosition
        });

        foreach (var item in OddNumbersWithIndexPosition)
        {
            Console.WriteLine($"IndexPosition :{item.IndexPosition} , Value :{item.Number}");
        }
        Console.ReadKey();
    }
}

```

Now run the application, and you will see the odd numbers and their index position, as shown below.

```

IndexPosition :0 , Value : 1
IndexPosition :2 , Value : 3
IndexPosition :4 , Value : 5
IndexPosition :6 , Value : 7
IndexPosition :8 , Value : 9

```

Let's rewrite the same example using LINQ Query Syntax.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<int> intList = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 }

            //Query Syntax

```

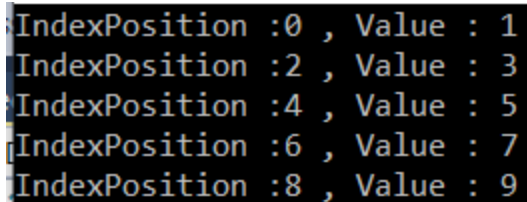
```

        var OddNumbersWithIndexPosition = from number in intList.Select((r
=> new {Numbers = num, IndexPosition = index })
        where number.Numbers % 2 != 0
        select new
        {
            Number = number.Numbers,
            IndexPosition = number.IndexPosition
        });

        foreach (var item in OddNumbersWithIndexPosition)
        {
            Console.WriteLine($"IndexPosition :{item.IndexPosition} , Value :{item.Number}");
        }
        Console.ReadKey();
    }
}

```

Now run the application, and it will also give the same output as the method syntax output, as shown in the below image.



```

IndexPosition :0 , Value : 1
IndexPosition :2 , Value : 3
IndexPosition :4 , Value : 5
IndexPosition :6 , Value : 7
IndexPosition :8 , Value : 9

```

Note: The Where method uses deferred execution, meaning the actual filtering of the collection doesn't occur until you iterate over the collection. This will improve the performance, especially when chaining multiple LINQ methods together.

Complex Example to Understand LINQ Where Extension Method using C#:

Let us see how to use the LINQ Where Extension Method with Complex Data Type using C#. We are going to use the following Employee class. So, create a class file named **Employee.cs** and copy and paste the following code. As we can see, we created the following Employee class with five properties, i.e., ID, Name, Gender, Salary, and Technology. Here, we have also created one method that will return the list of all employees which is going to our data source.

```

using System.Collections.Generic;
namespace LINQDemo
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Gender { get; set; }
        public int Salary { get; set; }
        public List<string> Technology { get; set; }
    }
}

```

```

public static List<Employee> GetEmployees()
{
    List<Employee> employees = new List<Employee>()
    {
        new Employee {ID = 101, Name = "Preety", Gender = "Female", Salary = 60000,
            Technology = new List<string>() { "C#", "Java", "ASP.NET" } },
        new Employee {ID = 102, Name = "Priyanka", Gender = "Female", Salary = 50000,
            Technology = new List<string>() { "WCF", "SQL Server", "ASP.NET" } },
        new Employee {ID = 103, Name = "Hina", Gender = "Female", Salary = 450000,
            Technology = new List<string>() { "MVC", "Java", "ASP.NET" } },
        new Employee {ID = 104, Name = "Anurag", Gender = "Male", Salary = 550000},
        new Employee {ID = 105, Name = "Sambit", Gender = "Male", Salary = 700000},
        new Employee {ID = 106, Name = "Sushanta", Gender = "Male", Salary = 700000,
            Technology = new List<string>() { "ADO.NET", "C#", "ASP.NET" } },
    };
    return employees;
}
}
}

```

Now, we need to fetch all the employees whose salary is greater than 50000. We need to use the LINQ Where Extension Method and specify the condition as **emp => emp.Salary > 50000** using Method Syntax and **employee.Salary > 50000** using Query Syntax, shown in the example below.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Query Syntax
            var QuerySyntax = from employee in Employee.GetEmployees()
                               where employee.Salary > 50000
                               select employee;

            //Method Syntax
            var MethodSyntax = Employee.GetEmployees()
                                   .Where(emp => emp.Salary > 50000);

            foreach (var emp in QuerySyntax)
            {
                Console.WriteLine($"Name : {emp.Name}, Salary : {emp.Salary}, {emp.Gender}");
            }
        }
    }
}

```

```

        if(emp.Technology != null && emp.Technology.Count() > 0)
        {
            Console.Write(" Technology : ");
            foreach (var tech in emp.Technology)
            {
                Console.Write(tech + " ");
            }
            Console.WriteLine();
        }
        else
        {
            Console.WriteLine(" Technology Not Available ");
        }
    }
    Console.ReadKey();
}
}
}

```

Output:

```

Name : Preety, Salary : 60000, Gender : Female
Technology : C# Java C++
Name : Anurag, Salary : 450000, Gender : Male
Technology Not Available
Name : Sambit, Salary : 550000, Gender : Male
Technology Not Available
Name : Sushanta, Salary : 700000, Gender : Male
Technology : ADO.NET C# LINQ

```

Example to Understand How to Specify Multiple Conditions using LINQ Where Method in C#

We need to fetch all the employees whose gender is Male and whose Salary is greater than 500000. So, here we have two conditions. The first condition is Gender = Male, and the second condition is Salary > 500000. If we have more than one condition, we must use **&& (AND)** or **|| (OR)** logical Operators based on our requirement. Here, we are going to use the AND (&&). That means if both the conditions are satisfied, then only return the data.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {

```

```

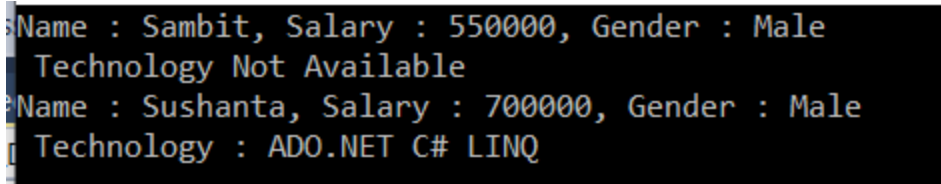
//Query Syntax
var QuerySyntax = from employee in Employee.GetEmployees()
                  where employee.Salary > 500000 && employee.Gender == Male
                  select employee;

//Method Syntax
var MethodSyntax = Employee.GetEmployees()
                    .Where(emp => emp.Salary > 500000 && emp.Gender == Male)
                    .ToList();

foreach (var emp in MethodSyntax)
{
    Console.WriteLine($"Name : {emp.Name}, Salary : {emp.Salary},
{emp.Gender}");
    if(emp.Technology != null && emp.Technology.Count() > 0)
    {
        Console.Write(" Technology : ");
        foreach (var tech in emp.Technology)
        {
            Console.Write(tech + " ");
        }
        Console.WriteLine();
    }
    else
    {
        Console.WriteLine(" Technology Not Available ");
    }
}
Console.ReadKey();
}
}
}

```

Output:



```

Name : Sambit, Salary : 550000, Gender : Male
Technology Not Available
Name : Sushanta, Salary : 700000, Gender : Male
Technology : ADO.NET C# LINQ

```

Complex Example to Understand LINQ Where Extension Method in C#:

Let us see a more complex example of the LINQ Where Extension Method in C#. Now, we will provide Multiple conditions with the custom operations and project the data to an anonymous type. The following is our business requirement.

We need to fetch all the employees whose salaries are greater than or equal to 50000, and technology should not be null. And we need to project the following information to an anonymous type.

1. Name as it is

2. Gender as it is
3. MonthlySalary = Salary / 12

The Complete Example Code is Given Below. In the example below, I show the Method and Query syntax with the Where Extension Method to achieve the above requirement.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Query Syntax
            var QuerySyntax = (from employee in Employee.GetEmployees()
                               where employee.Salary >= 50000 && employee.Technc
                               select new {
                                   EmployeeName = employee.Name,
                                   Gender = employee.Gender,
                                   MonthlySalary = employee.Salary / 12
                               }).ToList();

            //Method Syntax
            var MethodSyntax = Employee.GetEmployees()
                                   .Where(emp => emp.Salary >= 50000 && emp.Technc
                                   .Select(emp => new {
                                       EmployeeName = emp.Name,
                                       Gender = emp.Gender,
                                       MonthlySalary = emp.Salary / 12
                                   })
                                   .ToList();

            foreach (var emp in QuerySyntax)
            {
                Console.WriteLine($"Name : {emp.EmployeeName}, Gender : {emp.Gender}, Monthly Salary : {emp.MonthlySalary}");
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
Name : Preety, Gender : Female, Monthly Salary : 5000  
Name : Priyanka, Gender : Female, Monthly Salary : 4166  
Name : Sushanta, Gender : Male, Monthly Salary : 58333
```

Example to Fetch Elements along with their Index position using Where Method in C#

Now, we need to fetch all the employees whose Gender is Male and whose Salary is greater than 500000, along with their index position to an anonymous type.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Query Syntax
            var QuerySyntax = (from data in Employee.GetEmployees().Select((Data, index) => new { employee = Data, Index = index })
                               where data.employee.Salary >= 500000 &&
                               data.employee.Gender == "Male"
                               select new
                               {
                                   EmployeeName = data.employee.Name,
                                   Gender = data.employee.Gender,
                                   Salary = data.employee.Salary,
                                   IndexPosition = data.Index
                               }).ToList();

            //Method Syntax
            var MethodSyntax = Employee.GetEmployees().Select((Data, index) => new { employee = Data, Index = index })
                               .Where(emp => emp.employee.Salary >= 500000 &&
                               emp.employee.Gender == "Male")
                               .Select(emp => new
                               {
                                   EmployeeName = emp.employee.Name,
                                   Gender = emp.employee.Gender,
                                   Salary = emp.employee.Salary,
                                   IndexPosition = emp.Index
                               })
                               .ToList();

            foreach (var emp in QuerySyntax)
            {
                Console.WriteLine($"Position : {emp.IndexPosition} Name : {emp.EmployeeName}, Gender : {emp.Gender}, Salary : {emp.Salary}");
            }
        }
    }
}
```

```
        Console.ReadKey();  
    }  
}  
}
```

Output:

```
Position : 4 Name : Sambit, Gender : Male, Salary : 550000  
Position : 5 Name : Sushanta, Gender : Male, Salary : 700000
```

When should you use the LINQ Where Method in C#?

Here are some common situations where Where might be the right choice:

- **Filtering Data:** The most common use of Where is to filter a collection based on some criteria. For instance, you might want to find all products in a list that are cheaper than a certain price.
- **Combining with Other LINQ Methods:** The Where Method can be used in combination with other LINQ methods, like Select, OrderBy, GroupBy, etc., to perform complex queries on data. For example, you could filter a collection and then sort it or project it to a new form.
- **Deferred Execution:** LINQ queries, including those using the LINQ Where Method, benefit from deferred execution. This means the actual filtering doesn't happen until you iterate over the collection. This can be efficient when working with large data sets or when conditions change before the collection is iterated.
- **Readability and Maintainability:** Using LINQ Where Method makes your intentions clear and keeps your code readable. It's often more understandable than equivalent for each loop with if statements for filtering.
- **Working with In-Memory Collections and IQueryable Data Sources:** The Where Method is versatile and can be used both with in-memory collections like List<T> and Array, as well as IQueryable data sources like databases via Entity Framework. The syntax remains consistent, which simplifies coding across different data sources.
- **Conditional Filtering Based on Runtime Criteria:** If your filtering criteria can change at runtime (e.g., based on user input), Where is a great way to apply these dynamic conditions.
- **Chain Filtering Conditions:** You can chain multiple Where calls to apply multiple filters. Each call further refines the data set.
- **Performance Considerations:** For large collections, using Where can be more performant than manual looping, especially if chained with other LINQ methods that optimize the query execution.

In the next article, I will discuss the [LINQ OfType Operator using C#](#) with Examples. I hope this article gives you a very good understanding of the **LINQ Where Filtering Method in C#** with Examples.